

FOUNDATION_FILE_STORAGE - Binary Artifact Storage

Developer implementation guide: FOUNDATION_FILE_STORAGE_DEV_IMPL_GUIDE.md.

1. Purpose

This foundation defines how SOPHIX stores and serves binary artifacts: PDFs, images, signed documents, exports, attachments, KYC files, installation photos, and generated letters.

Use this document when implementing:

- Invoice PDF storage.
- KYC document upload.
- Work order or equipment photos.
- Signed agreement storage.
- Export file generation.
- Download APIs.
- Retention/lifecycle policies.

2. Ownership

FOUNDATION_FILE_STORAGE owns:

- Object storage technology choice.
- Bucket naming.
- Object key conventions.
- Access model.
- Retention/lifecycle defaults.
- Backup expectations.
- AWS S3 mapping.

FOUNDATION_FILE_STORAGE does not own:

- Document rendering. See NOT-01 and relevant billing DDs.
- API authorization rules for downloads. Each module owns its own access check.
- Database metadata schema. Each owning DD defines metadata fields.
- Virus scanning pipeline unless a later security DD introduces one.

3. Core Principle

Binary files live in object storage. Business tables store references to object keys.

Do not store large binary files in PostgreSQL.

Do not expose object storage directly to customers. Downloads must go through an application service that authenticates, authorizes, audits, and streams the object.

4. On-Prem Technology

Use MinIO, S3-compatible object storage, deployed per operator.

Item	Requirement
Protocol	S3 API
Deployment	One independent MinIO deployment per operator
Topology	2-node distributed MinIO cluster for medium start

Item	Requirement
Transport	Internal network; HTTPS when crossing network boundaries
Auth	MinIO IAM access key/secret per service
Customer direct access	Not allowed
Application access	Through service-specific credentials

Per-operator isolation:

Operator	Endpoint example
WIK	minio.wik.internal:9000
WUG	minio.wug.internal:9000
WTZ	minio.wtz.internal:9000
YAS SN	minio.yas-sn.internal:9000

Application code must resolve the endpoint from operator config. Do not hardcode operator endpoints in business logic.

5. Bucket Structure

Each operator deployment uses the same bucket names.

Bucket	Purpose	Main producers/consumers
invoices	Invoice, tax invoice, pro forma, credit/debit note PDFs	BIL-02, TAX-01, NOT-01, READ-01
kyc	Identity documents, proof of address, customer photos	ILM/KYC, FUL-02 KYC
equipment	Installation photos, CPE photos, condition evidence	WO-01, OSR, FUL-02
agreements	Signed contracts, T&C acceptance, change confirmations	Subscription workflows, ILM
exports	Reports, reconciliation files, tax exports	Billing, finance, audit
attachments	Generic ticket/dispute/admin attachments	Customer service, billing disputes
dunning	Dunning letters and collections artifacts	BIL-04, NOT-01

New top-level buckets require updating this foundation. Prefer prefixes inside existing buckets.

6. Object Key Convention

Pattern:

```
{domain_prefix}/{yyyy}/{mm}/{further_partition}/{filename}.{ext}
```

Examples:

```
invoices/2026/05/inv_01HZQX8N5V8KJEC9M3T2A4DPYW.pdf
invoices/2026/05/tax/taxinv_01HZQX8N5V8KJEC9M3T2A4D.pdf
invoices/2026/05/credit/cn_01HZQX8N5V8KJEC9M3T2A4D.pdf
kyc/cust_01HX4P5R6S7T8U9V/id_front_2026-03-15.jpg
equipment/2026/05/cpe_01HZ/install_front.jpg
agreements/cust_01HX/sub_01HY/signed_2026-04-01.pdf
exports/reconciliation/2026/05/wik_recon_2026-05-15.csv.gz
dunning/2026/05/dunning_01HZ.pdf
```

Rules:

Rule	Description
FILE-KEY-1	Keys are lowercase except platform IDs if case-sensitive.
FILE-KEY-2	Include year/month for lifecycle and browsing.
FILE-KEY-3	Use platform IDs, not customer names, phone numbers, ID numbers, or email addresses.
FILE-KEY-4	File extension must match content type.
FILE-KEY-5	Store relative object keys in DB metadata, not presigned URLs.
FILE-KEY-6	Object key must be immutable after published unless the owning DD defines a replacement/versioning flow.

7. Access Pattern

Customer/admin download path:

```
sequenceDiagram
    participant User as User/Browser
    participant API as SOPHIX API
    participant DB as Module DB
    participant OBJ as Object Storage

    User->>API: GET /api/invoices/{id}/pdf
    API->>API: Authenticate and authorize
    API->>DB: Load invoice metadata and object key
    API->>OBJ: Fetch object with service credentials
    API->>API: Audit download
    API-->>User: Stream file
```

Rules:

Rule	Description
FILE-ACCESS-1	No direct customer access to MinIO/S3.
FILE-ACCESS-2	No public buckets.
FILE-ACCESS-3	No customer-facing presigned URLs by default.
FILE-ACCESS-4	Application service must enforce ACL before fetching object.
FILE-ACCESS-5	Every successful download must be audit logged.
FILE-ACCESS-6	Each service gets scoped object-storage credentials.

8. Service Credentials

Use one object-storage IAM identity per service, scoped to minimum required buckets/actions.

Service	Buckets	Permissions
BIL-02 READ	invoices	Read
NOT-01	invoices, dunning	Read/write
ILM/KYC	kyc	Read/write
WO/Equipment	equipment	Read/write
Subscription service	agreements	Read/write
Billing/Finance exports	exports	Read/write
Customer service	attachments	Read/write

Credential rules:

- Store credentials in the platform secret manager.
- Do not share credentials across services.
- Rotate credentials through an ops runbook.
- Log access by service identity.

9. Java S3 Client Example

```
S3Client s3 = S3Client.builder()
    .endpointOverride(URI.create(operatorMinioEndpoint))
    .credentialsProvider(StaticCredentialsProvider.create(
        AwsBasicCredentials.create(accessKey, secretKey)))
    .region(Region.of("us-east-1"))
    .forcePathStyle(true)
    .build();

PutObjectRequest request = PutObjectRequest.builder()
    .bucket("invoices")
    .key("invoices/2026/05/inv_01HX.pdf")
    .contentType("application/pdf")
    .build();

s3.putObject(request, RequestBody.fromBytes(pdfBytes));
```

MinIO requires path-style access in many deployments; keep `forcePathStyle(true)` for on-prem MinIO unless the operator config says otherwise.

10. Metadata Pattern

Owning modules store object metadata in their own tables.

Example:

```
ALTER TABLE invoice
  ADD COLUMN pdf_bucket TEXT NULL,
  ADD COLUMN pdf_object_key TEXT NULL,
  ADD COLUMN pdf_content_type TEXT NULL,
  ADD COLUMN pdf_size_bytes BIGINT NULL,
  ADD COLUMN pdf_sha256 TEXT NULL,
  ADD COLUMN pdf_rendered_at TIMESTAMPTZ NULL;
```

Rules:

Rule	Description
FILE-META-1	Store bucket and object key separately.
FILE-META-2	Store content type.
FILE-META-3	Store size and checksum for critical legal documents.
FILE-META-4	Store render/upload timestamp.
FILE-META-5	Store object version ID if the backend supports versioning and the DD needs immutable versions.

11. Retention and Lifecycle

Default retention:

Bucket	Hot period	Archive/warm period	Delete after
invoices	2 years	Years 2-10	10 years
kyc	Active customer lifetime	7 years after closure	7 years post-closure
equipment	Equipment in service	2 years after retirement	2 years post-retirement
agreements	Active subscription lifetime	10 years after termination	10 years post-termination
exports	90 days	None	90 days
attachments	5 years	None	5 years
dunning	7 years	None	7 years

Operator legal/regulatory requirements override these defaults.

Lifecycle rules:

Rule	Description
FILE-LIFE-1	Lifecycle policies should delete/archive automatically where supported.
FILE-LIFE-2	Manual delete requires privileged storage admin role and audit log.
FILE-LIFE-3	Legal/tax document retention must be reviewed per operator before production.
FILE-LIFE-4	If hard immutability is required, enable object lock/versioning for the specific bucket/prefix.

12. Backup and Recovery

On-prem MinIO medium baseline:

- 2-node distributed MinIO gives single-node failure tolerance.
- Snapshot object storage to separate backup storage.
- Keep daily incremental, weekly full, monthly archive by operator policy.
- Test restore of representative files before go-live.

Rules:

Rule	Description
FILE-BACKUP-1	Backups must leave the MinIO nodes.
FILE-BACKUP-2	Backup scope is per operator; no cross-operator backups unless explicitly approved.
FILE-BACKUP-3	Restore tests must include legal PDFs and KYC images.
FILE-BACKUP-4	Store DB metadata and object storage backups consistently enough to recover references.

13. Failure Behavior

Failure	Behavior
One MinIO node down	Cluster continues in degraded mode. Restore node and let MinIO heal.
Both MinIO nodes down	File uploads/downloads fail. Business workflows that require files must pause or retry.
Object missing but DB metadata exists	Return controlled 404/ops error; create recovery ticket.
DB metadata missing but object exists	Object is orphaned; reconciliation job reports it.
Lifecycle deletes too much	Restore from backup if within backup retention.

14. Monitoring

Alert on:

Signal	Threshold
Node down	Any
Disk usage	Warning at 80 percent, critical at 90 percent
Failed PUT/GET rate	Sustained increase
p99 read latency	More than 500 ms sustained
Healing queue	Large or not decreasing
Lifecycle backlog	Sustained growth
IAM auth failures	Spike
Backup failure	Any

15. Medium Budget Infrastructure

15.1 Server Dimensions

Required medium baseline per operator:

Server role	Count	CPU	RAM	Disk	Notes
MinIO node	2	4 vCPU each	16 GB each	2-4 TB usable each	Distributed 2-node cluster. Size disk by retention and document volume.
Backup storage	1 shared target minimum	Depends	Depends	Larger than active data	NAS, backup server, tape, or operator cold storage.
Monitoring/log storage	Shared platform service	2 vCPU	4 GB	Depends	Metrics, logs, alerts.

Minimum acceptable start:

2 x MinIO nodes: 4 vCPU / 16 GB RAM / 2 TB SSD/HDD mix

Preferred medium:

2 x MinIO nodes: 4-8 vCPU / 16-32 GB RAM / 4 TB usable storage each

Storage note: SSD is best for latency, but object storage can use enterprise HDD for cost if PDF/image access volume is moderate. Keep metadata and OS on SSD if using mixed storage.

15.2 Recommended OS

Use one operator-standard Linux distribution:

OS	Recommendation
Ubuntu Server 22.04 LTS or 24.04 LTS	Recommended default for medium teams.
Rocky Linux 9 / RHEL 9	Use if the operator standardizes on RHEL-family systems.

OS baseline:

- 64-bit minimal server install.
- No desktop packages.
- chrony or equivalent NTP enabled.
- Security patching through operator maintenance process.
- SSH restricted to admin VPN/network.
- Host firewall enabled.

15.3 Disk Partitions

For each MinIO node with 2 TB disk:

```
/                40 GB
/opt/minio       20 GB
/var/log         40 GB
/data            1800 GB
/tmp             20 GB
free/reserved    remaining
```

For each MinIO node with 4 TB disk:

```
/                40 GB
/opt/minio       20 GB
/var/log         60 GB
/data            3700 GB
/tmp             30 GB
free/reserved    remaining
```

Partition rules:

- Store objects under /data or dedicated data mount.
- Do not store object data on /.
- Alert at 80 percent disk usage.
- Use xfs or ext4.
- Keep backup target separate from /data.

15.4 Network Ports

Flow	Port	Exposure
App service to MinIO S3 API	9000 or 443	Internal application network only
MinIO console	9001 or admin port	Admin VPN/network only
Admin SSH	22	Admin VPN/network only
Monitoring scrape	Operator standard	Monitoring network only
Backup copy	Operator standard	Backup network only

16. AWS Deployment Recommendations

This section applies only when deploying binary artifact storage on AWS.

16.1 Recommended AWS Shape

Use Amazon S3 instead of self-managed MinIO when deploying on AWS.

Component	AWS service	Recommendation
Object storage	Amazon S3	One bucket set per operator/environment.
Encryption	S3 server-side encryption	SSE-S3 by default; SSE-KMS for regulated buckets if required.
Private access	S3 VPC endpoint	Use gateway endpoint for private VPC access where applicable.
Access control	IAM roles and bucket policies	Service-specific IAM roles.
Lifecycle	S3 Lifecycle	Transition/archive/delete by prefix.
Audit	CloudTrail + S3 server access logs or CloudTrail data events where required	Enable for regulated buckets.

16.2 Bucket Mapping

AWS bucket naming pattern:

```
sophix-{env}-{operator}-{bucket}
```

Examples:

```
sophix-prod-wik-invoices
sophix-prod-wik-kyc
sophix-prod-wik-equipment
sophix-prod-wik-agreements
```

Alternative: one operator bucket with domain prefixes. Separate buckets give cleaner IAM and lifecycle boundaries, but more objects to administer.

Recommended for production: separate buckets for at least:

- invoices
- kyc
- agreements
- exports
- attachments/equipment/dunning as separate or grouped depending on ops preference

16.3 S3 Security

Required:

- Block Public Access enabled.
- Bucket policies deny public access.
- Server-side encryption enabled.
- IAM roles scoped per service and bucket/prefix.
- Access through VPC endpoint where possible.
- No long-lived AWS keys on application servers; use IAM roles.

Encryption:

Bucket	Recommendation
invoices	SSE-S3 minimum; SSE-KMS if legal/compliance requires key audit.
kyc	SSE-KMS recommended.
agreements	SSE-KMS recommended.
exports	SSE-KMS if finance/tax exports contain sensitive data.
Other buckets	SSE-S3 minimum.

16.4 S3 Lifecycle

Use S3 Lifecycle rules equivalent to the retention table in this DD.

Example lifecycle mapping:

SOPHIX phase	S3 storage class
Hot/recent	S3 Standard
Infrequent access	S3 Standard-IA or S3 Intelligent-Tiering
Long archive	S3 Glacier Flexible Retrieval or Glacier Deep Archive
Expiry	S3 Lifecycle expiration

Use Intelligent-Tiering where access patterns are unknown and object count/monitoring cost is acceptable.

16.5 AWS Network Layout

Recommended VPC layout:

```
Private app subnets:
- SOPHIX application services

S3:
- Regional managed service
- Access through VPC endpoint

Public subnets:
- No direct object storage exposure
```

Security controls:

Control	Recommendation
S3 VPC endpoint policy	Restrict to SOPHIX buckets.
Bucket policy	Allow only approved IAM roles and, where applicable, VPC endpoint source.
IAM	One role per service; least privilege.
CloudTrail	Enable data event logging for sensitive buckets if compliance requires.

16.6 AWS Operational Notes

- Do not generate public presigned URLs unless a DD explicitly approves that user flow.
- Prefer application streaming downloads for audit and access control.
- Enable versioning/Object Lock only for buckets that require immutability; this increases operational complexity.
- Use S3 Inventory or reconciliation jobs to detect orphaned/missing objects.
- Use replication to another region only if DR requirements justify cost.

Reference docs:

- [Amazon S3 documentation](<https://aws.amazon.com/documentation-overview/s3/>)
- [S3 server-side encryption](<https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingServerSideEncryption.html>)
- [S3 Lifecycle management](https://docs.aws.amazon.com/en_us/AmazonS3/latest/userguide/object-lifecycle-mgmt.html)
- [Gateway endpoints for Amazon S3](<https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints-s3.html>)

17. Developer Checklist

- [] File belongs in object storage, not PostgreSQL.
- [] Correct bucket selected.
- [] Object key follows naming convention.
- [] No PII in object key.
- [] Content type is set.
- [] Bucket/key metadata stored in owning module table.
- [] Size and checksum stored for legal/critical documents.
- [] Service credentials are scoped to required bucket/actions only.
- [] Download API enforces authorization before object fetch.

- [] Download is audit logged.
- [] Lifecycle/retention requirement is known for this file type.